Appendix C: IEEE 830 Template

Software Requirements Specification

For

campUStrade

Version 1.0

Prepared by Daniele Conte

Organisation: Lodz University of Technology

Date created: 17.04.2026

«Page Break»

Table of Contents

«Table of Contents»

Revision History

Name	Date	Reason for Changes	Version
Daniele Conte	17.04.2026	Initial Version, introduction and functionalities	1.0
Daniele Conte	17.04.2026	Adding the "Use Case Diagram" in 2.2 section and modifying the 2.3 section	1.1
Daniele Conte	20.04.2026	Adding the "UML Class Diagram"	1.2

«Page break»

1. Introduction

1.1. Purpose

«Identify the product whose software requirements and design are specified in this document, including the revision or release number. Describe the scope of the product that is covered by this specification.»

The purpose of this document is to specify the software requirements of the “campUStrade” platform and to introduce it: the scope of this document covers the core functionalities necessary for the initial release, specifically focusing on user registration, the creation of service listings, and the management of the virtual credit transaction system between students.

1.2. Document Conventions

«Describe any standards or typographical conventions that were followed when writing this specification, such as fonts or highlighting that have special significance. For example, state whether priorities for higher-level requirements are assumed to be inherited by detailed requirements, or whether every requirement statement is to have its own priority.»

1.3. Intended Audience and Reading Suggestions

«Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. Describe what the rest of this document contains and how it is organized. Suggest a sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each reader type.»

1.4. Product Scope

«Provide a short description of the software being specified and its purpose, including relevant benefits, objectives, and goals. Relate the software to corporate goals or business strategies when applicable.»

University students have to deal with many financial problems, making it difficult to afford professional services such as academic tutoring, laptop repairs or language practice. At the same time, universities are an environment rich in human capital, where thousands of individuals possess diverse and highly valuable skills. The primary problem is the absence of a centralized, secure, and moneyless platform to make easier the exchange of these services among them.

"campUStrade" is a web-based software designed to solve this problem by introducing a virtual credit-based economy. The system allows students to offer their specific skills to earn virtual credits, which they can spend to request services from other students as well.

By removing a real currency, the product aims to foster a collaborative community, making essential services accessible to all students regardless of their financial situation.

1.5. References

«List any other documents or Web addresses to which this document refers or was based on. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location. Include the assignment document in the references.»

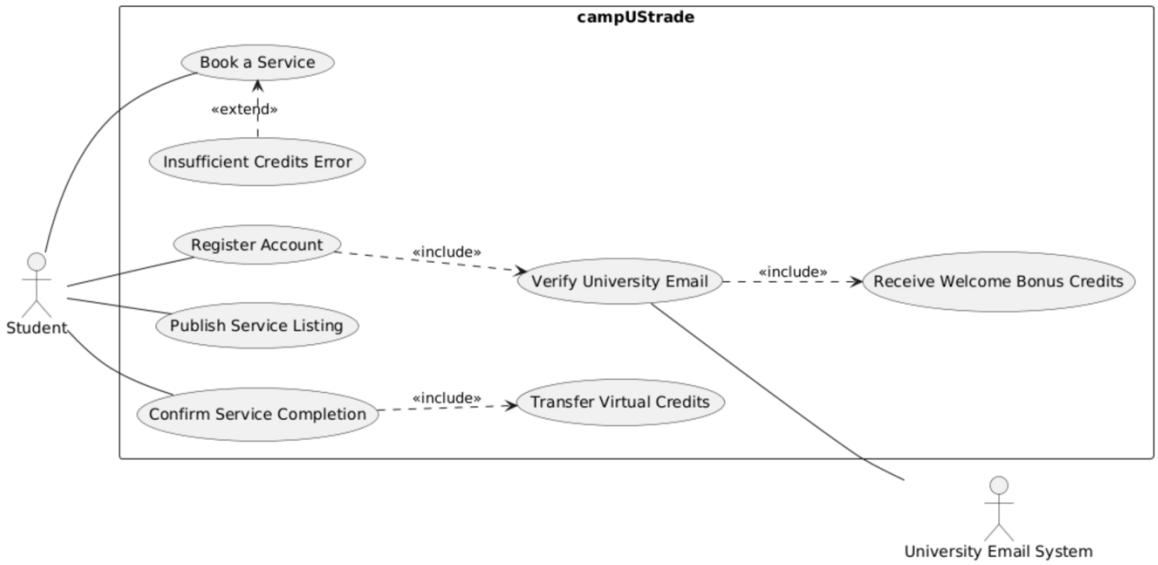
2. Overall Description

2.1. Product Perspective

«Describe the context and origin of the product being specified in this specification. For example, state whether this product is a follow-on member of a product family, a replacement for certain existing systems, or a new, self-contained product.»

2.2. Product Features

The campUStrade platform provides several core features to allow a moneyless exchange of services among students. The following Use Case Diagram illustrates these major functionalities and how the users interact with the system:



Summary of the main features:

- Account Registration: Students register using their university email. The system automatically verifies the email and grants a welcome bonus.
- Service Listing: Users can publish offers for their specific skills.
- Service Booking: Users can spend their virtual credits to book services from others, provided they have sufficient funds.
- Service Completion: Upon confirmation of a finished job, the system securely transfers the virtual credits from the buyer to the seller.

2.3. User Classes and Characteristics

The campUStrade platform is designed primarily for one specific user class, with a secondary administrative class for maintenance:

- Student (Primary User Class): The main target audience for this product. These are university students looking to exchange services. They must possess a valid university email address. They interact with all the main features of the system (registering, publishing, booking, and confirming services).
- System Administrator (Secondary User Class): A technical user responsible for monitoring the platform, resolving disputes between students, and maintaining the system. They have high technical expertise and advanced privilege levels to manage users and virtual credits if things go wrong.

2.4. Operating Environment

«Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.»

2.5. Design and Implementation Constraints

«Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).»

2.6. User Documentation

«List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.»

2.7. Assumptions and Dependencies

«List any assumed factors (as opposed to known facts) that could affect the requirements stated in the specification. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).»

3. System Features

«The purpose here is to give a brief introduction to the system features, so that §4 has something to reference. Then, the full explanation of the system features is given in §5, likely referencing §4. The numbering in §3 and §5 should match.»

3.1. «Feature Name»

«Brief description of the feature, generally 1-3 sentences»

3.«N». «Additional feature blocks as needed»

4. External Interface Requirements

4.1. User Interfaces Overview

«Describe the high level functionality of the system from the user's perspective. Explain how the user should be able to use your system to complete all the expected features and the feedback information that will be displayed for the user (for each screen the user will see). For example, specify whether it is GUI or text based interface and how at high level it would work. Discuss at high level how user will interact with the game such as clicking buttons, typing in some selection character, etc.»

«Describe the minimum requirements for the interface. This may include some sample screen images whether for text or GUI approach, any GUI standards or product family style guides that are to be followed, screen layout constraints, for GUI standard buttons and functions (e.g., help) that must appear on every screen, keyboard shortcuts, error message display standards, and so on»

4.2. Hardware Interfaces

«Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.»

4.3. Software Interfaces

«Describe the connections between this product and other specific software components (name and version), including databases, interpreters, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Identify data that will be shared across software

components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.»

5. System Features/Modules

«This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product.»

5.1. «System Feature Name»

5.1.1 Description and Priority

«Provide a short description of the feature. Give its priority, too.»

5.1.2 Stimulus/Response Sequences

«"Stimulus" or "Condition"»: «Describe the stimulus or condition»

Response: «Describe the response»

«Additional Stimulus/Response pairs as needed»

5.1.3 Functional Requirements

«Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous, verifiable, and necessary. Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.»

FR-01: The system shall allow users to register an account strictly using a valid university email address (e.g., email ending in @edu.p.lodz.pl).

FR-02: The system shall allow a user to publish a service listing containing a title, a text description, and a specific cost in virtual credits.

FR-03: The system shall deduct the specified amount of virtual credits from the buyer's account immediately upon booking a service.

FR-04: The system shall transfer the held virtual credits to the seller's account only after the buyer clicks the "Service Completed" button.

FR-05: The system shall prevent a user from booking any service if their current virtual credit balance is strictly lower than the service's credit cost.

FR-06: The system shall allocate 10 virtual credits to the account balance of every newly registered user upon successful email verification.

6. Nonfunctional Requirements

6.1. Performance

NFR-01: The system shall process and reflect virtual credit transfers between user accounts in less than 1.5 seconds.

NFR-02: The system shall allow a new user to complete the account registration process in less than 2 minutes.

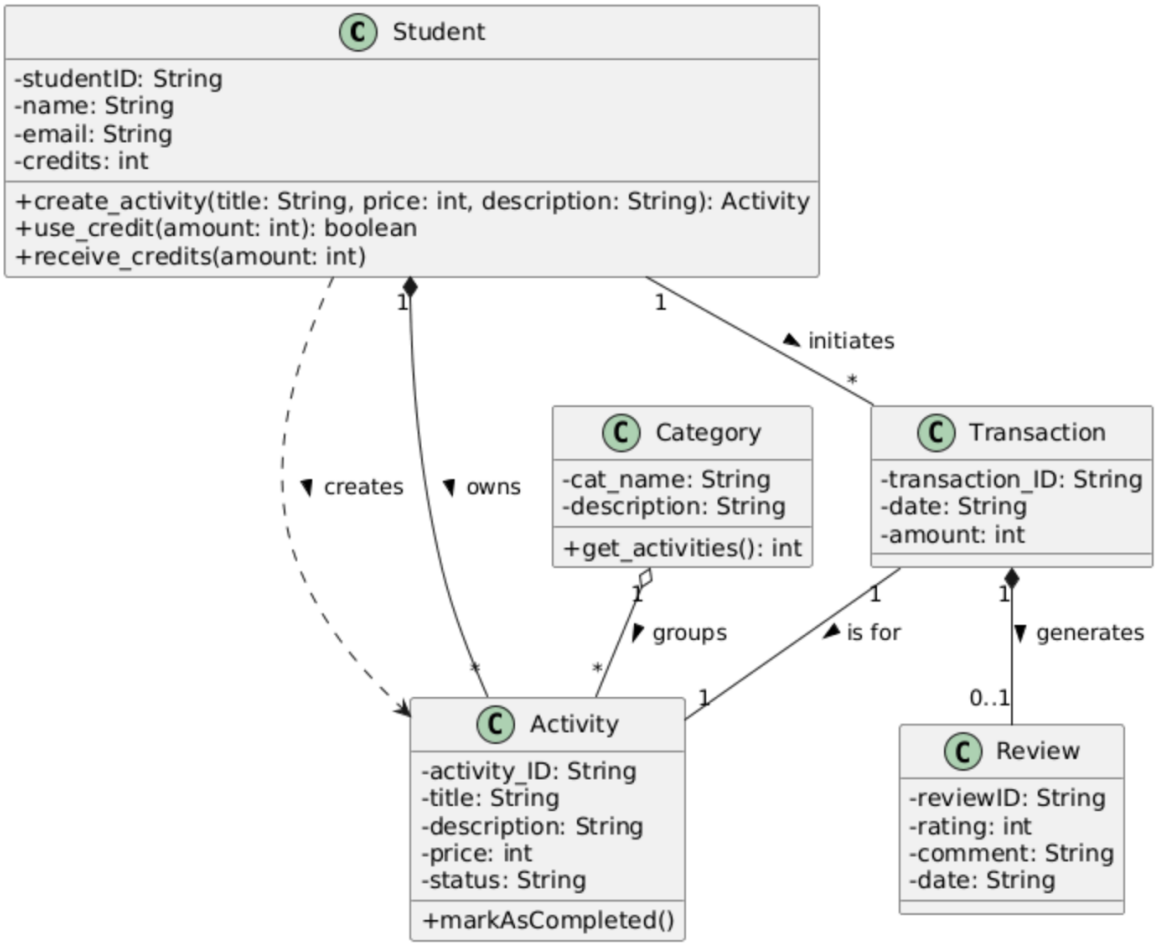
6.2. Security

NFR-03: All user passwords shall be hashed using the bcrypt algorithm before being stored in the database.

7. System Design: UML Class Diagram

7.1 Overview: This section illustrate the data architecture of campUStrade in a UML Class Diagram, which enlights the entities and their private attribute, methods and the relations between them.

7.2 Class Diagram



7.3 Class Description

The system is divided in 5 core classes:

- Student: Represents the system user. It encapsulates the virtual currency balance (-credits) and provides secure methods for financial interactions, such as “+use_credit()” and “+receive_credits()”.
- Activity: Represents a service or task offered by the student (e.g., tutoring for “Database Programming”).
- Category: Logically groups various activities to improve searchability and platform organization.
- Transaction Acts as a passive historical record that tracks credit exchanges between a student and a specific activity.
- Review: Manages the feedback system (rating and comments) generated upon the successful completion of a transaction. A student must leave a feedback so it can help other students to choose their activities more wisely.

7.4 Design Decisions & Relationships

To ensure data integrity and proper object lifecycle management, the following relationships were implemented:

- Composition (Student -- Activity): A strong "part-of" relationship. If a Student account is deleted, all associated Activities are removed to maintain a correct order in the database.
- Composition (Transaction -- Review): A Review cannot exist without a completed Transaction. Deleting the transaction record automatically removes the associated feedback.
- Aggregation (Category o-- Activity): A weak "has-a" relationship. Deleting a Category does not destroy the Activities within it; they are simply unassigned or moved to another category.
- Dependency (Student ..> Activity): The Student class depends on the Activity class specifically for the creation process, as shown in the “+create_activity()” method.